



Bioinformatics

Ing. Giulia Fiscon, PhD

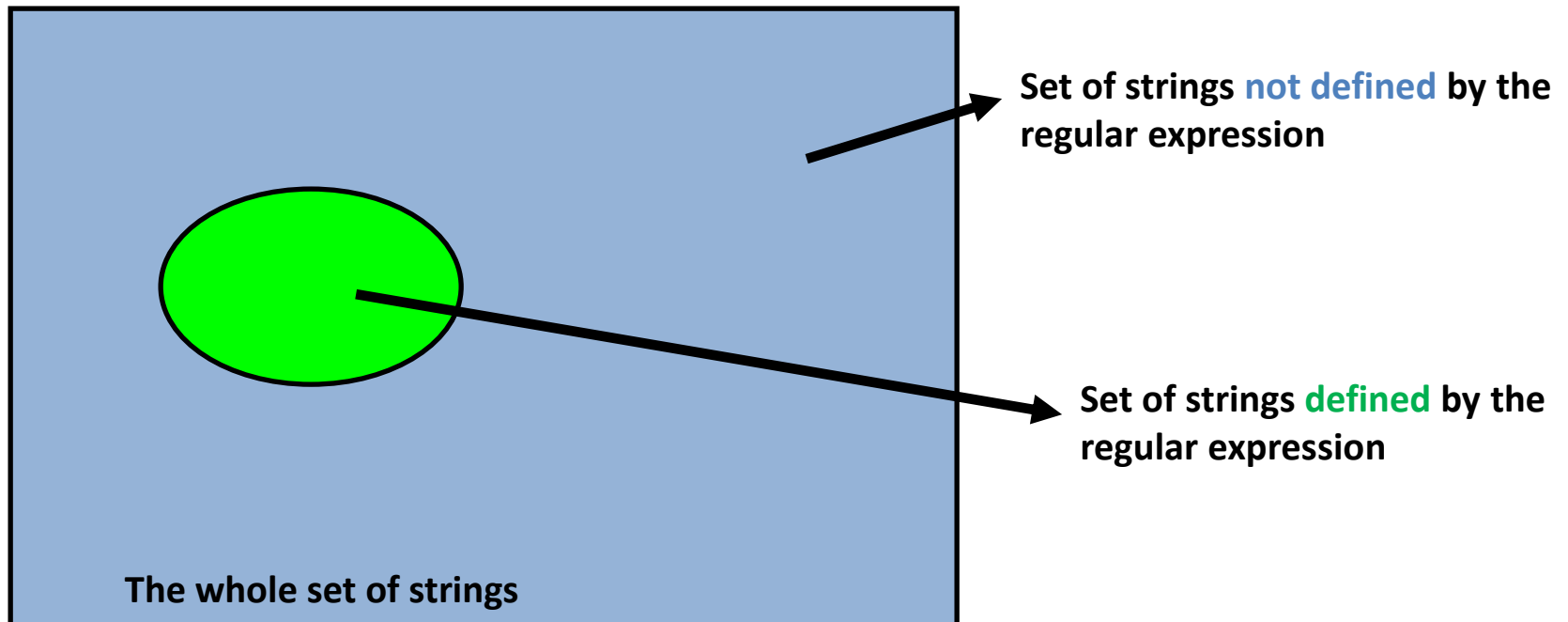
fiscon@diag.uniroma1.it

Department of Computer, Control, and Management Engineering
Sapienza University of Rome

⬡ Regular expressions

Pattern matching

Searching for **patterns** within strings, i.e. special substrings described through the **regular expression** language



Alphabet

- ❑ A regular expression is a sequence of character, **pattern**, that can match repeating characters or strings, and alternative characters or strings
- ❑ The repetition and alternation are expressed using **metacharacters** (such as *****, **+**, **|**)
- ❑ Regular expressions are so called due to the type of the language (grammar), “regular” that allows to define them
- ❑ The strings that a regular expression can match are sequences of symbols drawn from an alphabet

Alphabet

- ❑ This alphabet is a set of characters that can be divided in:
 - **ordinary characters** that match themselves. They include letters, digits, whitespace (such as spaces, tabs, and newlines), and other characters that are not metacharacters
 - **metacharacters** that are themselves characters. It's used for escaping with a backslash `\` to distinguish when a metacharacter is being used as a regular character rather than as a metacharacter with special meaning:
 - ✓ `*` is a metacharacter, but `\*` represents an asterisk character
 - ✓ `\` is the escape metacharacter, but `\\` represents a single backslash character
- ❑ Escaping is also used to give some regular characters a special meaning:
 - `\n` represents a newline and allows a newline to be matched explicitly in pattern languages

Metacharacters

.	matches any character (except new-line)
^	matches the beginning of the line; moreover, before a group denies it
\$	matches the end of the line
 	Alternation (OR condition)
[]	matches a single character that is contained within the brackets, you can use a dash symbol "-" to match a character between two extremes (e.g. [a-z] equal to a letter from a to z or [0-9] equal to a number from 0 to 9)
\	Escape the next metacharacter
\n	matches the new-line character (start a new line)
()	Grouping: round brackets identify groups of characters
\t	matches the tabular character (tab)

Metacharacters: ^ and \$

^ The caret symbol indicates that the pattern must be present at the beginning of the string (Not to be confused with ^ within square brackets)

^sea matches all the strings beginning with "sea": "sea", "season", "seaside", "seashore",... but not "aseatic" that would be recognized by sea (without ^).

\$ Indicates that the pattern must be present at the END of the string

less\$ matches all the strings ending with "less": "less", "meaningless", "heartless", ... but not "lesson", "voicelessness" that would be recognized by less (without \$)

Metacharacters: |

	allows to choose by two or more alternatives (OR condition)
pippo pluto pflip	matches all the strings that contain "pippo", "pluto" or "pflip", for example even " <u>pluto</u> n", "My name is <u>pippo</u> "
cas(e eness)	matches all the strings that contain "case" or "caseness". Equal to write case(ness)?
sadd?(ism le)	matches all the strings that contain "sad", "sadism", "saddle"
	(? Indicates that the preceding item is optional and will be matched at most once)

Metacharacter: []

Examples:

[aeiou] matches a vowel

[oi] matches "o" or "i"

[ht]ype matches all the strings that contain **ype** preceded by h or t (e.g., "**hype**", "**type**", but also "sub**type**", "**hypersensitive**", "**typeset**")

[a-z] matches a lowercase letter

[A-Z] matches an UPPERCASE letter

[a-zA-Z] matches a letter

[0-9] matches a number (equal to **[0123456789]** or **[4859013726]**)

Metacharacter: [^]

[^] matches a character other than those indicated in brackets

[^ 0-9] matches any non-numeric character

[^ A-zA-Z] matches any non-alphabetic character

Other symbols:

\d matches a number (= [0-9])

\D matches a character other than a digit (= [^ 0-9])

\w matches an alphanumeric character (= [a-zA-Z0-9_])

\W matches a non-alphanumeric character (= [^ a-zA-Z0-9_])

\s matches a "spacing" character (spaces, new-line, tab, ...)

\S matches a non-spacing character

Metacharacter \

Escape character \ allows to insert a metacharacter as a regular character

\\	matches a backslash \
\ 	matches a pipe symbol
\+	matches a plus sign +
*	matches an asterisk *
\?	matches a question mark ?
\.	matches a dot .
\(\)	matches round brackets ()
\[\]	matches squared brackets []
\{ \}	matches curly brackets {}
\\(\\w+\\.\\w+\\)	matches a patter composed of two alpha-numeric strings, joint by a dot "." and enclosed in round brackets, e.g.: (123.AF63)

Quantifiers

- Quantifiers specify how many instances of a character, group, or character class must be present in the input for a match to be found

*	matches 0 or more occurrences; equal to {0,}
+	matches 1 or more occurrences; equal to {1,}
?	matches 1 or 0 occurrences; equal to {0,1}
{N}	Finds exactly N occurrences;

Remember that curly braces are considered regular characters in all other contexts

{N,}	Searches for minimum N occurrences (at least N)
{N,M}	Searches for minimum N and maximum M occurrences (at least N at most M)
{0,M}	Searches for maximum M occurrences

The quantifier: *

- *** The star symbol indicates that the previous character or regular expression (you can use the brackets) can be repeated **0 or more times**
- sat20*** Matches "sat2" but also "sat20", "sat200", "123sat231", that is anything that contains "sat2" followed by any number of 0 (even none)
- [0-9]*** Matches a sequence of 0 or more digits, so any natural number or empty string
- ma*mma** Matches "mma", "mamma", "maamma", "maaamma", ...

The quantifier: +

+	The plus sign indicates that the previous character or regular expression (you can use the brackets) can be repeated 1 or more times
Sat20+	Matches <u>"sat20"</u> but also <u>"sat200"</u> , <u>"sat2000"</u> , <u>"123sat2031"</u> that is anything that contains "sat2" followed by at least a zero
[0-9]+	Matches a sequence of 1 or more digits, so any natural number
ma+mma	Matches <u>"mamma"</u> , <u>"maamma"</u> , <u>"maaamma"</u> , ...
{ }	If we know the exact repetitions number of the previous character or expression, this number can be indicated within { }
e.g.: ca{3}se	Look for everything that contains "caaase"

Quantifier: ?

?

The character or regular expression (you can use brackets) is optional. Check for 0 or 1 occurrence of the preceding character.

go(ne)?

Matches "gone" but also "go", "forgone", "algorithm" (i.e., all the strings containing "gone" or "go")

Penn?y?

Matches "penny", "pen" but also "independent", (i.e., all the strings containing "pen" or "penn" or "penny")

[1-2]? \d \d \d

Matches a number of 3 or 4 digits.

If it is 4 digit number, the first digit is 1 or 2

Exercises

Write the regular expressions matching:

At the beginning of the string, the symbol greater than (>) followed by any number of white spaces (also 0) and an alphanumeric code of at least 1 character

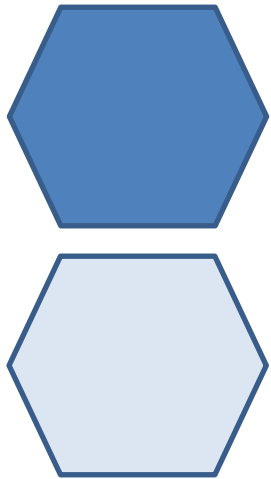
`^> *\w+`

At any point in the string, the string "surname" immediately followed by colon symbol : and by at least one blank space. Then, it follows a person's surname

`surname: +[A-Za-z '\-]+`

At any point in the string, or the string AGE: followed by any number of blank spaces (also 0) and a number or the string GENDER: followed by any number of blank spaces (also 0) and M or F

`(AGE: *\d+) | (GENDER: *[MF])`



Regular expressions



Regular expressions in R

- R supports the concept of regular expressions, which allows you to search for patterns inside text.
- Regular expressions allow three ways of making a search pattern more general than a single, fixed expression:
 - **Alternatives:** You can search for instances of one pattern or another, indicated by the `|` symbol.

For example `beach|beech` matches both *beach* and *beech*.

- **Grouping:** You group patterns together using brackets `( )`.

For example you write `be(a|e)ch` to find both *beach* and *beech*.

- **Quantifiers:** You specify whether an element in the pattern must be repeated or not by adding `*` (occurs zero or many times) or `+` (occurs one or many times).

For example, to find either *bach* or *beech* (zero or more of *a* and *e* but not both), you use `b(e*|a*)ch`.

Grep function

- **Grep function** takes as input the **pattern to search** (encoded as regular expression) and **the strings** in which searches for the pattern
- By default it returns as output the indices of the matching, it returns instead the names by setting argument: `value = TRUE`

Example:

First, create a new variable with five words:

```
> rwords <- c("bach", "back", "beech", "beach", "black")
```

Find either beach or beech using alternative matching:

```
> grep("beach|beech", rwords)
[1] 3 4
```

This means the search string was found in elements 3 and 4 of rwords.

To extract the actual elements, you can use subsetting with square brackets or `value = TRUE`:

```
> rwords [grep("beach|beech", rwords)]
[1] "beech" "beach"
```

Now use the grouping rule to extract the same words:

```
> rwords [grep("be(a|e)ch", rwords)]
[1] "beech" "beach"
```

Lastly, use the quantifier modification to extract bach and beech but not beach:

```
rwords [grep("b(e*|a*)ch", rwords)]
[1] "bach" "beech"
```

Escape character in R

Metacharacters and how to escape them in R

Metacharacter	Literal meaning	Escape in R
.	the period or dot	\\.
\$	the dollar sign	\\\$
*	the asterisk or star	*
+	the plus sign	\\+
?	the question mark	\\?
	the vertical bar or pipe symbol	\\
\\	the backslash	\\\\
^	the caret	\\^
[	the opening square bracket	\\[
]	the closing square bracket	\\]
{	the opening curly bracket	\\{
}	the closing curly bracket	\\}
(	the opening round bracket	\\(
)	the closing round bracket	\\)